

Project
Artificial Intelligence

Sudoku application implemented with Native Interface Design (natID)

Authors:

Andrijana Kešelj 20008

Andela Maksimović 20009

Azra Kurić 20113

Professor:

Prof. Dr. Izudin Džafić

February 16th, 2026.

Sarajevo

Contents

1	Introduction	2
2	Backtracking Algorithm	2
2.1	Overview	2
2.2	Problem Representation	2
2.3	Constraint Enforcement and Candidate Generation	3
2.4	Variable Selection Strategy	3
2.5	Recursive Backtracking Procedure	3
2.6	State Preservation and Supporting Functions	4
2.7	Algorithm Characteristics	4
3	Hints Engine	5
3.1	Implemented Solving Techniques	5
3.2	Fallback Mechanism	7
4	Logical representation of Sudoku	7
5	Insertion	8
6	Generation with Difficulty Levels	8
7	Game Mode	9
8	How to play	10
9	Additional Features	10

1 Introduction

This project presents a full Sudoku game built in C++ using the NatID GUI framework. It combines classic puzzle gameplay with clean software design, creating an enjoyable experience for both casual players and dedicated Sudoku fans. With intelligent puzzle generation, an advanced hint system, and a carefully designed interface, the game highlights practical uses of algorithms, data structures, and interactive application development.

The game includes a smart puzzle generator that creates valid Sudoku boards across multiple difficulty levels, each designed to require different solving techniques. A backtracking solver with optimization heuristics ensures that every puzzle is valid and has a unique solution. The hint system goes a step further by helping players learn solving strategies through interactive explanations and visual guidance. Overall, the project places a strong emphasis on user experience, featuring customizable themes, intuitive controls, background music, and polished visual feedback that make solving puzzles both smooth and enjoyable.

2 Backtracking Algorithm

2.1 Overview

The Backtracking Solver is a deterministic search algorithm that models Sudoku as a Constraint Satisfaction Problem (CSP). It is executed when the user starts a game and functions primarily as a validation and support mechanism. The solver verifies puzzle solvability, checks solution uniqueness when required, and provides fallback hints when rule-based techniques cannot proceed.

Rather than imitating human reasoning strategies, the algorithm performs a systematic search of the solution space. It incrementally assigns values to empty cells while enforcing Sudoku constraints. If a contradiction occurs, the solver reverses the most recent assignment and explores an alternative. This guarantees that a solution will be found if one exists.

2.2 Problem Representation

The Sudoku grid is represented using:

- **Variables:** 81 cells
- **Domains:** Values 1–9 for each empty cell
- **Constraints:**
 - Unique values per row
 - Unique values per column
 - Unique values per 3×3 subgrid

A valid solution is a complete assignment satisfying all constraints.

2.3 Constraint Enforcement and Candidate Generation

Before assigning a value, the solver checks whether the placement violates row, column, or subgrid constraints. For each empty cell, the algorithm computes the set of permissible values under the current board state. If no valid values exist, the current search branch is abandoned immediately.

2.4 Variable Selection Strategy

To improve efficiency, the solver applies the Minimum Remaining Values (MRV) heuristic. The Minimum Remaining Values heuristic selects the empty cell with the fewest valid candidate numbers, reducing branching by addressing the most constrained position first. This strategy reduces branching and exposes contradictions earlier in the search process, significantly improving practical performance despite the exponential worst-case complexity.

2.5 Recursive Backtracking Procedure

The solver operates recursively:

1. Select an empty cell using MRV.
2. Try each valid candidate value.
3. Recursively continue solving the remaining grid.

4. If a contradiction occurs, undo the assignment and try the next value.

The algorithm terminates successfully when all cells are filled without violating constraints.

2.6 State Preservation and Supporting Functions

To prevent permanent modification of the game state, the solver stores and restores the board when performing auxiliary tasks. This mechanism supports:

- Solvability verification
- Solution uniqueness checking (bounded enumeration)
- Hint generation via full-solution reference

2.7 Algorithm Characteristics

The method is complete and deterministic. While the theoretical time complexity is exponential, constraint pruning and MRV-based selection make solving tractable for standard Sudoku instances.

3 Hints Engine

The Hint Engine is a rule based system designed to assist Sudoku solving by identifying logically deducible cell values from the current board state. It operates by computing candidate values for each empty cell according to row, column, and subgrid constraints and then evaluating a predefined sequence of solving techniques arranged from simpler to more complex. A hint is generated when a technique allows the elimination of alternatives such that a single value can be placed with certainty.

For each identified move, the system returns the target cell, the resolved value, an explanation of the reasoning, and contextual data used for visual highlighting in the user interface. When no implemented technique yields a deterministic placement, the system optionally derives a move from a completed solution produced by a backtracking solver. The following section describes the individual solving techniques applied by the engine.

3.1 Implemented Solving Techniques

The engine evaluates solving techniques in a predefined order of increasing complexity and prioritizes the simplest explanation that produces a deterministic placement.

Level 1: Naked Single

- **Detection:** A cell with exactly one valid candidate.
- **Method:** All values already present in the cell's row, column, and subgrid are excluded. If a single candidate remains, it is placed.
- **Message example:** “[Naked Single] Place 7 in R2C8 - it's the only possible value here.”

Level 2: Hidden Singles (Row/Column/Box)

- **Detection:** A number that can only go in one specific cell within a row, column, or box
- **How it works:** For each missing value in a row, column, or subgrid, the engine evaluates all empty cells in that unit. If only one cell satisfies the constraints, the value is placed there.

- **Message example:** “[Hidden Single] Place 4 in R5C3 - only spot for 4 in row 5.”

Level 3: Naked Pairs

- **Detection:** Two cells in a unit that contain the exact same two candidates
- **Method:** When two cells contain identical candidate sets of size two, those values are restricted to those cells and removed from other cells in the same unit. If this elimination produces a single candidate elsewhere, that value is placed.
- **Message example:** “[Naked Pair] Place 5 in R3C7 - cells R3C1 and R3C5 lock values {2,7} in row 3.”

Level 4: Pointing Pairs/Triples (Box-Line Reduction)

- **Detection:** All candidate positions for a value within a subgrid lie in a single row or column.
- **Method:** When a value’s candidates in a subgrid align within one row or column, that value is eliminated from the remaining cells of that row or column outside the subgrid. If a single candidate remains in a cell, it is placed.
- **Message example:** “[Pointing Pair] Place 5 in R2C3 - value 5 points to row 2 from box 2.”

Level 5: Box/Line Reduction (Claiming)

- **Detection:** When all candidates for a number in a row/column fall within a single box
- **Method:** If a number in a row can only fit in cells that belong to one box, eliminate it from other cells in that box.
- **Message example:** “[Box/Line] Place 6 in R4C5 - value 6 claims row 4 in box 5.”

Level 6: Naked Triples

- **Detection:** Three cells that share exactly three candidates between them

- **Method:** Similar to naked pairs but with three cells and three numbers. These three numbers are locked to those three cells.
- **Message example:** “[Naked Triple] Place 3 in R1C5 - cells R1C2, R1C4, R1C8 lock values {1,5,9} in row 1.”

Level 7: Hidden Pairs

- **Detection:** Two numbers that can only appear in the same two cells within a unit
- **How it works:** If two numbers in a row/column/box can only fit in the same two cells, those cells can’t contain any other numbers.
- **Message example:** “[Hidden Pair] Place 3 in R6C2 - values {3,8} only fit in R6C2 and R6C7 in row 6.”

Level 8: X-Wing

- **Detection:** A rectangular pattern where a candidate appears in exactly 2 cells in 2 rows (or columns), and these cells align perfectly
- **How it works:** Forms a rectangle pattern. If a number appears in the same 2 columns in exactly 2 rows, it can be eliminated from other cells in those columns.
- **Message example:** "Number 5 appears only in columns 3 and 7 in rows 2 and 8, forming an X-Wing. Eliminate 5 from other cells in columns 3 and 7."

3.2 Fallback Mechanism

If no human technique applies, the engine uses the **BacktrackingSolver** to find the next valid move. This ensures the hint system always provides a valid next step, even for extremely difficult puzzles.

4 Logical representation of Sudoku

The **SudokuBoard** class represents the core logical data model of the application, encapsulating the complete 9×9 grid structure and all related cell state information. Each cell

stores its current value, an indicator of whether it belongs to the initial puzzle configuration, and a set of candidate notes. The class provides controlled methods for value assignment, validation of placements according to Sudoku rules, note management, board clearing, and solved-state detection. By isolating puzzle logic from the user interface and solving algorithms, this design ensures clean separation of concerns, maintainability, and extensibility within the overall system architecture.

5 Insertion

The Board Insertion feature is implemented through coordination between the `ViewHome` controller and the `SudokuBoard` data model. When the user activates insertion mode via the Insert button, the board is fully cleared using `SudokuBoard::clearAll(true)`, which removes all values and resets initial flags. The application then enables insert mode in the canvas, allowing unrestricted value placement without validation against Sudoku rules.

During insertion, values entered by the user are written directly to the board without mistake tracking, timer constraints, or notes support. Upon finishing insertion, the system validates that at least 17 numbers are present, ensuring a potentially solvable puzzle. All non-zero cells are then re-marked as initial using `setValue(..., true)`, making them read-only for gameplay. Insert mode is disabled, the timer is reset, and the board transitions into standard game mode.

This design cleanly separates puzzle creation from puzzle solving, while reusing the same board structure and ensuring consistency with the normal gameplay flow.

6 Generation with Difficulty Levels

Sudoku puzzle generation is implemented in the `GenerateSudoku` class using a multi-phase strategy that guarantees both solvability and difficulty control. First, a fully solved board is created by filling diagonal 3×3 boxes randomly and completing the grid using a `BacktrackingSolver`. Once a valid solution is obtained, clues are removed strategically while continuously verifying that the puzzle maintains a unique solution.

Difficulty is not determined solely by the number of clues, but by analyzing which solving techniques are required to complete the puzzle. The generator simulates logical solving using the `HintEngine`, tracks which techniques are necessary such as Naked Singles, Hid-

den Singles, Naked Pairs, Box-Line Reduction, X-Wing, or Backtracking, and determines the maximum technique level required. Each difficulty level enforces specific constraints: Easy puzzles require only basic techniques, Medium puzzles require intermediate strategies, and Hard puzzles require advanced logical deductions.

Through repeated attempts, the generator selects the best puzzle that satisfies both uniqueness and the targeted technique complexity. All remaining non-zero values are then marked as initial, producing a playable puzzle whose logical depth aligns with the selected difficulty level.

7 Game Mode

Game Mode represents the standard interactive solving phase of the application and begins once a puzzle is generated or manually inserted and the user presses the **Start Game** button. At this moment, the system computes the full solution using the **BacktrackingSolver**, storing it internally for validation, hint generation, and the “Show Number” functionality. The timer is activated, mistake tracking is enabled, and the board transitions into active gameplay.

All predefined values, whether produced by the generator or entered through Board Insertion, are marked as *initial* in the **SudokuBoard** data model. These cells are displayed in black and remain locked throughout the game to preserve the original puzzle structure. Any values entered by the player during gameplay are treated as non-initial and are visually distinguished in pink, clearly separating user input from fixed clues.

During Game Mode, users can select cells, enter values, manage candidate notes, and request hints. Every insertion is validated against Sudoku rules using the stored backtracking solution, enabling accurate mistake detection and logical hint support. The system continuously monitors progress and automatically detects when the puzzle has been successfully completed.

Game Mode integrates the **SudokuBoard** data model, the internally stored solution, the **HintEngine**, and UI state management within **ViewHome**, ensuring controlled gameplay while maintaining clean separation between puzzle logic, validation mechanisms, and presentation layers.

8 How to play

How to play view provides an interactive educational interface that explains fundamental Sudoku concepts, including rows, columns, and 3×3 boxes. The `ViewInfo` class manages navigation through topics using Previous and Next buttons, while the `InfoCanvas` model dynamically renders topic-specific images, titles, and instructional text. The layout divides the screen into a visual explanation area and a structured message panel with styled text and graphics, ensuring clarity and readability. This design separates presentation control from drawing logic, maintaining modularity while offering users a guided learning experience within the application.

9 Additional Features

- **Notes Mode:** Pencil in candidate numbers to track possible values in empty cells
- **Mistake Tracking:** Three-strike system with visual and audio feedback for incorrect placements
- **Number Highlighting:** Automatically highlights all instances of a selected number on the board
- **Keyboard Input:** Enter numbers directly from your keyboard for faster gameplay
- **Clear All:** Quickly reset all user-entered numbers while preserving the initial puzzle
- **Visual Themes:** Choose from Pink Dream, Hot Pink, Lilac Mist, and Coral Reef color schemes
- **Background Music:** Three focus-enhancing music tracks or silent mode
- **Timer:** Track solve time with pause and resume functionality
- **Game Over Display:** Special visual feedback when mistakes limit is reached
- **Victory Celebration:** Magenta border around the board and achievement sound when puzzle is solved
- **Insert Sudoku:** Manually enter custom puzzles from external sources (minimum 17 clues)
- **Save for Later:** Export your current puzzle state to continue another time

- **Load Board:** Import previously saved puzzles
- **Show Number:** Reveal the solution for a selected cell when stuck
- **Solution:** Display the complete solution
- **How To Play:** Provided explanations for sudoku rules
- **PonchoGirl:** Added informative messages by Poncho Girl, and hints powered by:
 - **Technique Recognition:** Identifies and teaches Naked Singles, Hidden Singles, Naked Pairs, Pointing Pairs, and more advanced strategies
 - **Visual Guidance:** Highlights relevant cells, rows, columns, and boxes to illustrate each technique